

VORBEREITUNG ZUR VORLESUNG

Handout 04 – Container-Netzwerke und Datenbanken

Modul 3IT-VSIT-50 · Verteilte Systeme und Internet der Dinge
Thema Docker-Netzwerke, Service Discovery, Datenbank-Container
Dozent Dr. Ulrich Winkler
Semester 5. Semester · Informationstechnik

Mehrere Container, die zusammenarbeiten

Echte Anwendungen bestehen selten aus einem einzigen Prozess. Typisch ist eine **Anwendung plus Datenbank**. In der Container-Welt heißt das: mehrere Container, die miteinander reden müssen. Dafür gibt es **Docker-Netzwerke**.

Das Bridge-Netzwerk

Legen Sie ein Netzwerk an, verbindet Docker die daran angeschlossenen Container über eine virtuelle Brücke:

```
docker network create lognetz
docker run --network lognetz ...
```

Container im selben Netzwerk erreichen sich, Container in verschiedenen Netzwerken sind voneinander isoliert. Das ist zugleich ein Sicherheitsgewinn: Nur was zusammengehört, kann sich sehen.

Service Discovery: Container finden sich über den Namen

Der entscheidende Komfort: Docker bringt in jedem selbst angelegten Netzwerk einen **eingebauten DNS** mit. Ein Container erreicht einen anderen **über dessen Namen**, nicht über eine IP-Adresse. Heißt der Datenbank-Container `db`, dann verbindet sich unser `log_tool` einfach zum Host `db`:

```
psycopg.connect(host="db", dbname="logbuch", user="logger", ...)
```

Häufiger Fehler

„could not translate host name db“, bedeutet fast immer: Die Container sind nicht im selben Netzwerk. Prüfen Sie das `-network` bei jedem `docker run`.

Ports: EXPOSE vs. -p

Zwei Dinge, die oft verwechselt werden:

- **Innerhalb** eines Docker-Netzwerks reden Container direkt miteinander – dafür ist **kein** `-p` nötig. `log_tool` erreicht `db` auf Port 5432 allein durch die Netzwerkzugehörigkeit.
- **`-p 8080:8080`** veröffentlicht einen Port nach außen, damit Sie ihn vom Browser (bzw. in Codespaces über die Portweiterleitung) erreichen. Deshalb bekommt **Adminer** ein `-p`, die Datenbank für den internen Zugriff dagegen nicht.

Die Datenbank ist einfach ein Container

`postgres:17` ist ein fertiges Image. Man gibt ihm per Umgebungsvariablen Datenbank, Benutzer und Passwort mit (`POSTGRES_DB` , `POSTGRES_USER` , `POSTGRES_PASSWORD`) und ein Volume für die Daten – fertig ist ein vollwertiger PostgreSQL-Server. Kein lokales Installieren, kein Konfigurieren.

Adminer: das Fenster in die Daten

Adminer ist ein winziges Web-Werkzeug, mit dem Sie sich per Browser an der Datenbank anmelden und Tabellen ansehen. In den Übungen kontrollieren Sie damit, dass Ihre `log_tool` -Einträge wirklich in der Tabelle `logs` landen.

Wann ist die Datenbank bereit?

Ein frisch gestarteter PostgreSQL-Container braucht beim allerersten Mal einen Moment, bis er Verbindungen annimmt. Verbindet sich `log_tool` zu früh, gibt es einen Verbindungsfehler. In Übung 04 warten Sie deshalb kurz mit `pg_isready` ; ab Übung 05 erledigt das ein **Healthcheck** in Docker Compose automatisch.